

언어에서 벡터로: 텍스트 표현과 문서 유사도의 기하학적 이해

자연어 처리의 기초부터 TF-IDF,
그리고 코사인 유사도 기반 추천 시스템까지



정부기관정보공개법...
정보공개법...
정보공개법...

기계가 단어를 인식하는 두 가지 근원적 방법론

	국소 표현 (Local Representation)	분산 표현 (Distributed Representation)
개념	단어 자체만 보고 특정 수치를 맵핑(Mapping)하는 방식	주변 단어를 참고하여 해당 단어를 정의하는 방식
다른 명칭	이산 표현 (Discrete Representation)	연속 표현 (Continuous Representation)
예시	Puppy = 1, Cute = 2, Lovely = 3	Puppy 주변에 Cute, Lovely가 자주 등장 → Puppy는 귀엽고 사랑스러운 뉘앙스를 가짐
한계	단어의 의미나 뉘앙스를 표현할 수 없음	
발전형	본 강의 범위: Bag of Words, DTM, TF-IDF	Word2Vec, FastText, GloVe (토마스 미코로브의 연속 표현 포괄 개념 포함)

NLP Evolution

순서를 버리고 빈도에 집중하다: Bag of Words (BoW)

단어의 순서는 전혀 고려하지 않고, 오직 **출현 빈도(Frequency)**에만 집중하는 텍스트 데이터 수치화 기법.



Bag of Words 생성 2단계

Step 1: 단어 집합(Vocabulary) 생성

각 단어에 고유한 정수 인덱스 부여



Step 2: 빈도수 벡터(Vector) 기록

각 인덱스 위치에 단어 토큰의 등장 횟수를 기록

활용처: 문서의 성격 판단, 분류, 유사도 측정 (예: 미분, 방정식 등장 빈도가 높으면 수학 문서로 분류)

파이썬과 KoNLPy를 활용한 한국어 BoW 구현

○ ○ ○

```
tokenized_document = okt.morphs(document)

if word not in word_to_index.keys():
    # Add new word to dictionary
else:
    # Increment word frequency
```

Output and Execution Results

입력: 정부가 발표하는 물가상승률과 소비자가
느끼는 물가상승률은 다르다.

Vocabulary:
{'정부': 0, '가': 1, '발표': 2, '하는': 3,
'물가상승률': 4, '과': 5, ...}

BoW Vector:
[1, 2, 1, 1, 2, 1, 1, 2, 1, 1, 1, 1]

인덱스 4번('물가상승률')이
2회 언급되어 5번째 값이
2로 기록됨

패키지 기반 BoW 생성 (CountVectorizer)과 불용어 처리

영어 BoW 예제

입력: you know I want your love. because I love you.

출력 Vocabulary: {'you': 4, 'know': 1, 'want': 3, 'your': 5, 'love': 2, 'because': 0}

특징: 길이가 1인 알파벳 **I**는 토큰화 과정에서 자동 제거됨

한국어 적용 시의 한계

단순 띄어쓰기 기준으로 작동하여, '물가상승률과', '물가상승률은'을 서로 다른 단어로 인식

불용어 (Stopwords) 제거 3가지 패턴

1. 사용자 직접 정의

```
stop_words=['the', 'a', 'is', 'not']
```

2. 자체 지원 불용어

```
stop_words='english'
```

3. NLTK 라이브러리 연동

```
stopwords.words('english')
```

다수 문서의 결합: 문서 단어 행렬 (DTM)과 본질적 한계

	과일이	길고	노란	먹고	바나나	사과	싶은	저는	좋아요
문서1 (먹고 싶은 사과)	0	0	0	1	0	1	1	0	0
문서2 (먹고 싶은 바나나)	0	0	0	1	1	0	1	0	0
문서3 (길고 노란 바나나 바나나)	0	1	1	0	2	0	0	0	0
문서4 (저는 과일이 좋아요)	1	0	0	0	0	0	0	1	1

1. 희소 표현 (Sparse Representation)

전체 코퍼스 방대 시 차원의 저주 발생.
행렬의 대부분이 0으로 채워져 공간적 낭비 및 계산 복잡도 극증.

2. 단순 빈도수 기반 접근

'the'와 같은 불용어가 자주 등장한다고 해서
문서 간 유사도가 높다고 오판할 위험성 내포.

가중치의 발견: TF-IDF (Term Frequency - Inverse Document Frequency)

불용어에는 패널티를, 특정 문서에만 등장하는 핵심 단어에는 가점을 부여하는 가중치 산출 시스템.

TF (단어 빈도, $tf(d, t)$)

특정 문서 d 에서 특정 단어 t 가 등장한 횟수 (DTM의 기존 값)



DF (문서 빈도, $df(t)$)

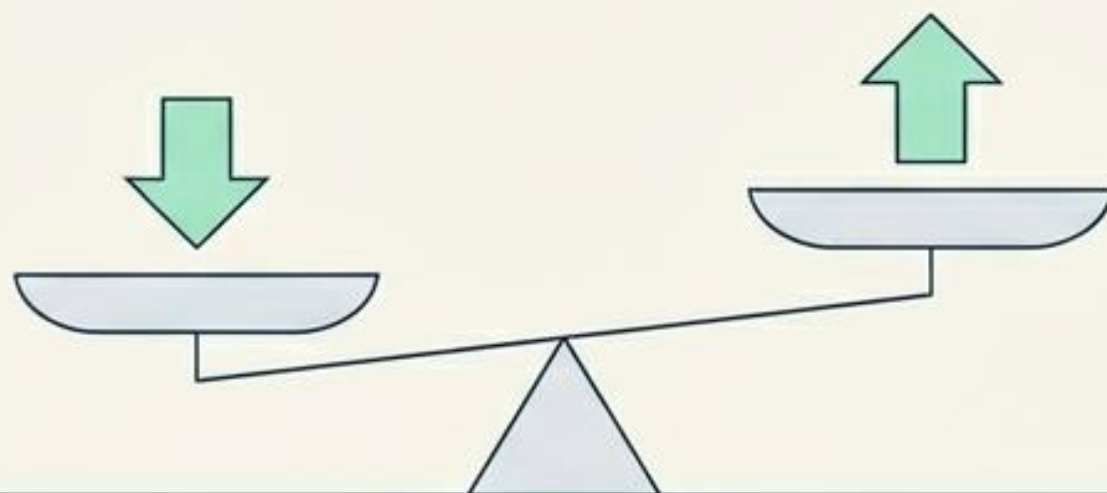
특정 단어 t 가 등장한 '문서의 수'



IDF (역 문서 빈도, $idf(t)$)

DF의 역수 개념.
 $\log(n / (df(t) + 1))$

모든 문서에 자주 등장
→ 중요도(가중치) 하락



특정 문서에만 집중 등장
→ 중요도(가중치) 상승

수리적 시각화: 왜 IDF 공식에는 Log(로그)가 필수적인가?

총 문서의 수(n) = 1,000,000

Log를 씌우지 않았을 때

단어 1 (1개 문서 등장) = 가중치 1,000,000

단어 6 (1,000,000개 문서 등장) = 가중치 1

문제점: 희귀 단어에 지나치게 방대한
가중치(100만 배) 부여

Log (밑 10)를 씌웠을 때 (실제 IDF)

단어 1 (1개 문서 등장) = 가중치 6

단어 6 (1,000,000개 문서 등장) = 가중치 0

해결책: 값의 격차를 획기적으로 줄여
연산의 안정성 확보

Zero-division 방지: 분모에 +1을 하여 분모가 0이 되는 오류 원천 차단

파이썬으로 직접 구현하는 TF-IDF 연산 시뮬레이션

```
def tf(t, d):  
    return d.count(t)  
  
def idf(t):  
    return log(N/(df+1))  
  
def tfidf(t, d):  
    return tf(t,d) * idf(t)
```

Aha Moment: '바나나'의 가중치 변화

공통 IDF 값 연산:

$$\ln(4 / (2+1)) = 0.287682$$

문서2의 '바나나' TF-IDF:

$$1 \times 0.287682 = 0.287682$$

문서3의 '바나나' TF-IDF:

$$2 \times 0.287682 = 0.575364$$

Pandas 출력 결과

문서1: 0.693147 (사과)

문서2: 0.287682 (바나나)

문서3: 0.575364 (바나나)

실무 환경의 TF-IDF: Scikit-learn TfidfVectorizer

기본 식의 수학적 맹점

총 문서 수가 4개이고 DF가 3일 때, 분모(3+1)=4가 되어 $\ln(4/4) = 0$ 발생. 가중치 역할 상실.

Scikit-learn의 수식 보정 메커니즘

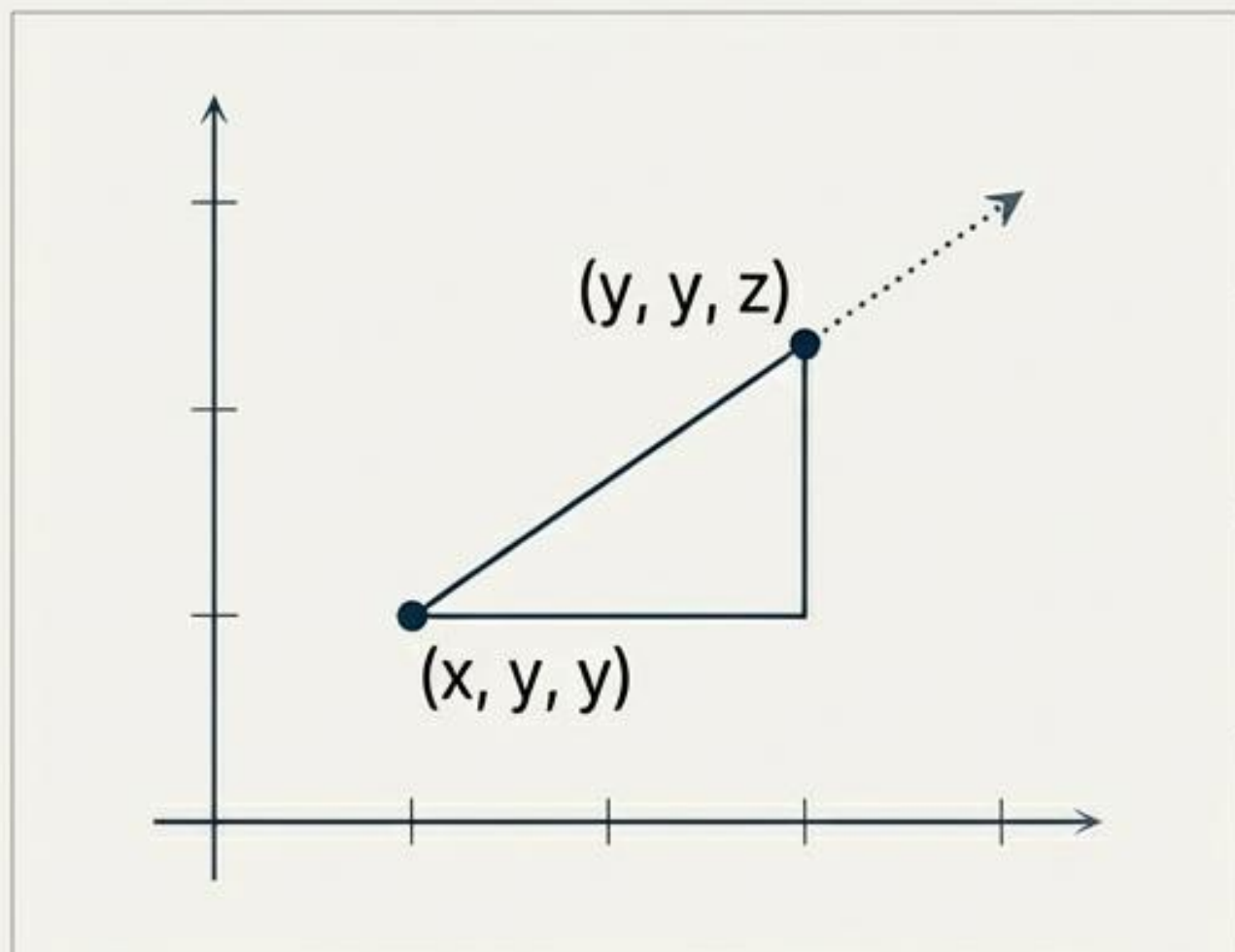
- 로그항 분자와 분모 모두에 1을 더하는 등 내부적인 식 조정
- L2 정규화(Normalization)를 통해 행렬 값을 0~1 사이로 안정화

Python 실행 코드

```
tfidf = TfidfVectorizer().fit(corpus)
```

출력된 어레이 예시: [0. 0.467 0. 0.467 ... 0.355 0.467]

기하학적 관점의 문서 유사도 1: 유클리드 거리 (Euclidean Distance)



다차원 공간에서의 직선 거리 측정

Python 구현 공식:

```
np.sqrt(np.sum((x-y)**2))
```

Execution Results

비교 실험: 기준 문서Q [1, 1, 0, 1]

문서1 - 문서Q 거리: 2.236 (유사도 높음)

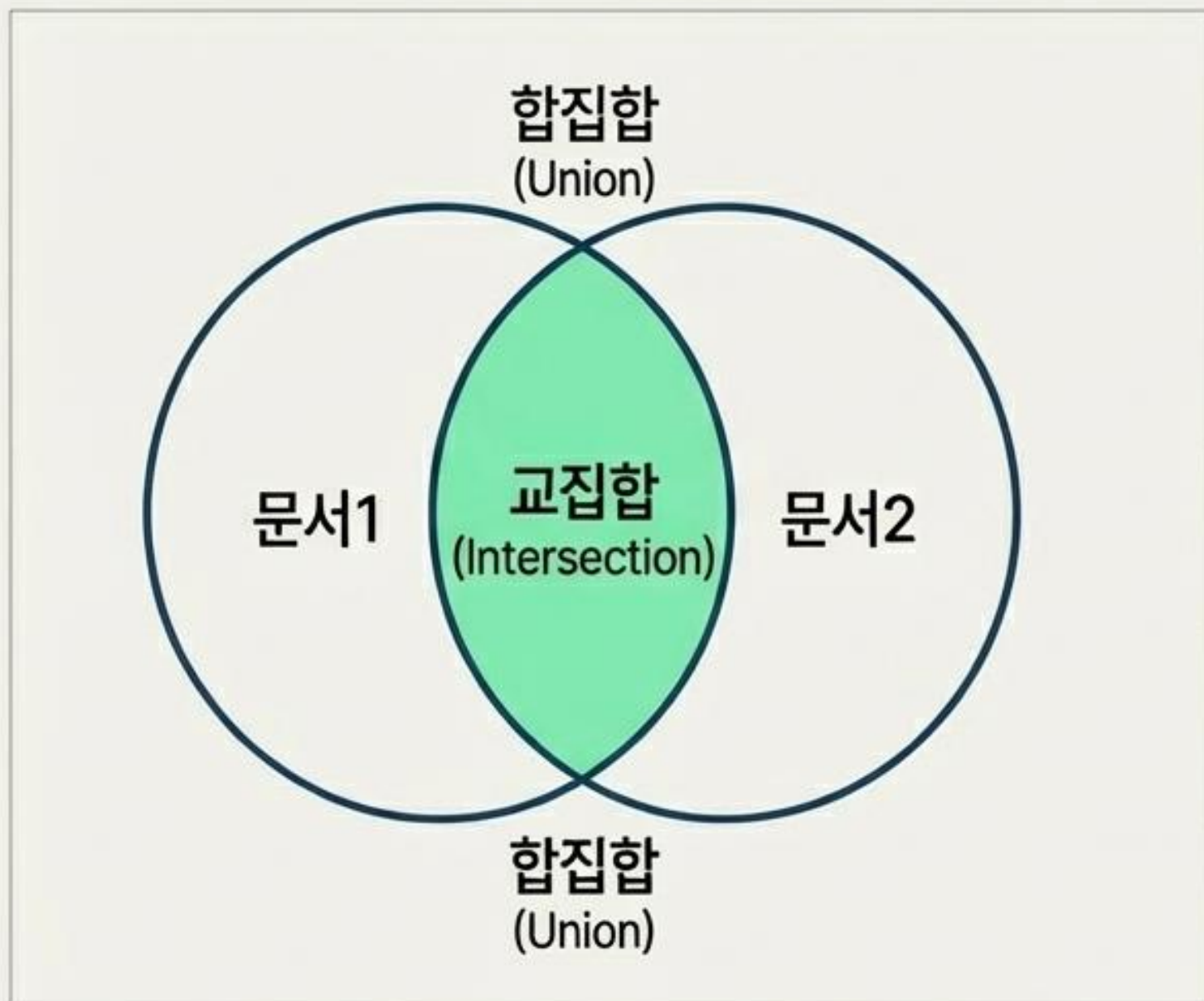
문서2 - 문서Q 거리: 3.162

문서3 - 문서Q 거리: 2.449

한계점: 유클리드 거리는 직관적이나, 문서의 길이(단어 빈도 총량)에 너무 큰 영향을 받음.

기하학적 관점의 문서 유사도 2: 자카드 유사도 (Jaccard Similarity)

합집합의 크기 대비 교집합의 크기 비율로 유사도를 측정 (0~1 사이의 값)



집합 연산을 시각화한 벤 다이어그램

Python 집합(Set) 연산

문서1: apple banana everyone like
likey watch card holder

문서2: apple banana coupon
passport love you

합집합 (Union): 12개 원소 (apple, banana, everyone, like, likey, watch, card, holder, coupon, passport, love, you)

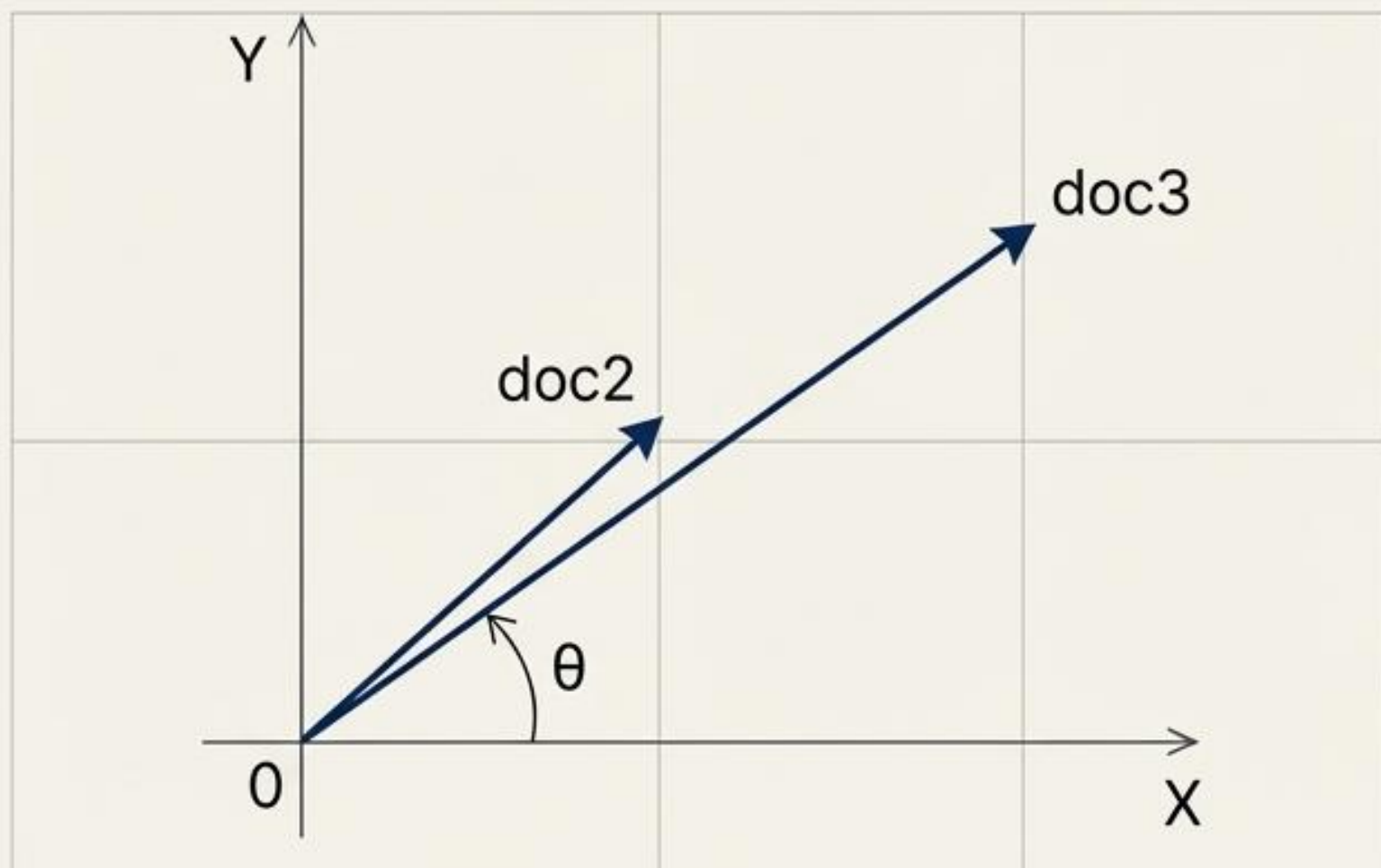
교집합 (Intersection): 2개 원소 (apple, banana)

산출 결과 (자카드 유사도)

$$2 / 12 = 0.1666...$$

문서 유사도의 최종 해답: 코사인 유사도 (Cosine Similarity)

두 다차원 벡터 간의 코사인 각도를 연산. 길이가 달라도 패턴(방향)이 같으면 동일한 문서로 간주.



두 벡터의 각도를 통한 유사도 측정 (방향성 일치)

왜 코사인 유사도인가? (Why Cosine Similarity?)

`doc2 = [1, 0, 1, 1]`

`doc3 = [2, 0, 2, 2]`

수식 적용 결과:

`cos_sim(doc1, doc2) = 1.00`

결론: 문서의 절대적 길이가 달라도, 단어 사용의 비율(방향성)이 일치하면 완벽히 유사한 문서로 판별.

실전 응용: Kaggle 영화 줄거리 기반 추천 시스템 구축 (1단계)

데이터셋 로드 및 전처리

- 소스: Movies Metadata (상위 **20,000개** 추출)
- 피처: 영화 제목(title)과 줄거리(overview)
- 전처리: 결측치(Null) 135개 발견 → `data['overview'].fillna('')`로 빈 값 대체

TF-IDF 행렬 생성 및 형상(Shape) 분석

```
tfidf = TfidfVectorizer(stop_words='english')  
tfidf_matrix.shape → (20000, 47487)
```

해석: **20,000개**의 영화 줄거리를 수치화하기 위해, **47,487 차원**의 거대한 문서 벡터 공간이 구축되었음을 의미.

실전 응용: 코사인 유사도 연산 및 추천 시스템 엔진 구동 (2단계)

추천 시스템 핵심 알고리즘

1. 입력된 영화 제목의 인덱스 추출

2. `cosine_similarity(tfidf_matrix, tfidf_matrix)` 연산

3. 유사도 스코어 매핑 후 내림차순 정렬

4. **Top 10** 최우수 유사 영화 인덱스 추출 및 제목 반환

시스템 실행 결과

입력: `The Dark Knight Rises`

출력: `The Dark Knight, Batman Forever, Batman Returns, Batman Begins` 등

최종 결론: 텍스트의 국소적 표현을 **TF-IDF**로 가중치화하고 **코사인 각도**로 방향성을 측정함으로써, 기계는 줄거리만으로 인간의 주제와 장르를 완벽하게 **기하학적으로 이해**해냈다.